

Proyecto 1 – Etapa 1

Sistema de gestión de pedidos y reseñas de restaurantes

Descripción del problema y alcance

El proyecto consiste en diseñar e implementar un sistema backend basado en MongoDB que permita gestionar restaurantes, usuarios, artículos de menú, pedidos y reseñas. La solución debe permitir:

- Registrar restaurantes con ubicación geoespacial
- Gestionar artículos de menú
- Realizar pedidos con múltiples items
- Dejar reseñas asociadas a pedidos
- Generar reportes analíticos
- Visualizar dashboards en Tableau

El sistema será implementado utilizando MongoDB Atlas con menú en consola para comprobar su funcionamiento.

Supuestos

- Un usuario puede realizar múltiples pedidos
- Un restaurante puede tener muchos artículos
- Cada orden está asociada a un único restaurante
- Una reseña puede estar asociada a un pedido específico
- Los artículos de un menú pertenecen exclusivamente a un restaurante, no hay compartidos entre restaurantes
- Los usuarios no podrán modificar órdenes una vez estas estén en estado “ENTREGADO”
- Las imagenes asociadas a artículos serán almacenadas en GridFS y no directamente en los documentos
- Se asume que la colección ordenes será la de mayor crecimiento, por lo que será la elegida para implementar sharding

Modelo de datos detallado con diagramas de documentos

Colección: Restaurantes

{

Grupo 12

```
_id: ObjectId,  
nombre: String,  
descripcion: String,  
categoria: String,  
ubicacion: {  
  type: "Point",  
  coordinates: [Number, Number]  
},  
direccion: String,  
telefono: String,  
ratingPromedio: Number,  
totalReseñas: Number,  
fechaCreacion: Date,  
activo: Boolean  
}
```

Colección: Usuarios

```
{  
  _id: ObjectId,  
  nombre: String,  
  email: String,  
  passwordHash: String,  
  direccion: String,  
  telefono: String,  
  fechaRegistro: Date,  
  rol: String,  
  totalGastado: Number  
}
```

Grupo 12

Colección: MenuItem

```
{  
  _id: ObjectId,  
  restaurantId: ObjectId,  
  nombre: String,  
  descripcion: String,  
  precio: Number,  
  categoria: String,  
  disponible: Boolean  
}
```

Colección: Ordenes

```
{  
  _id: ObjectId,  
  userId: ObjectId,  
  restaurantId: ObjectId,  
  items: [  
    {  
      menuItemId: ObjectId,  
      nombre: String,  
      precioUnitario: Number,  
      cantidad: Number,  
      subtotal: Number  
    }  
  ],  
  total: Number,  
  estado: String,  
}
```

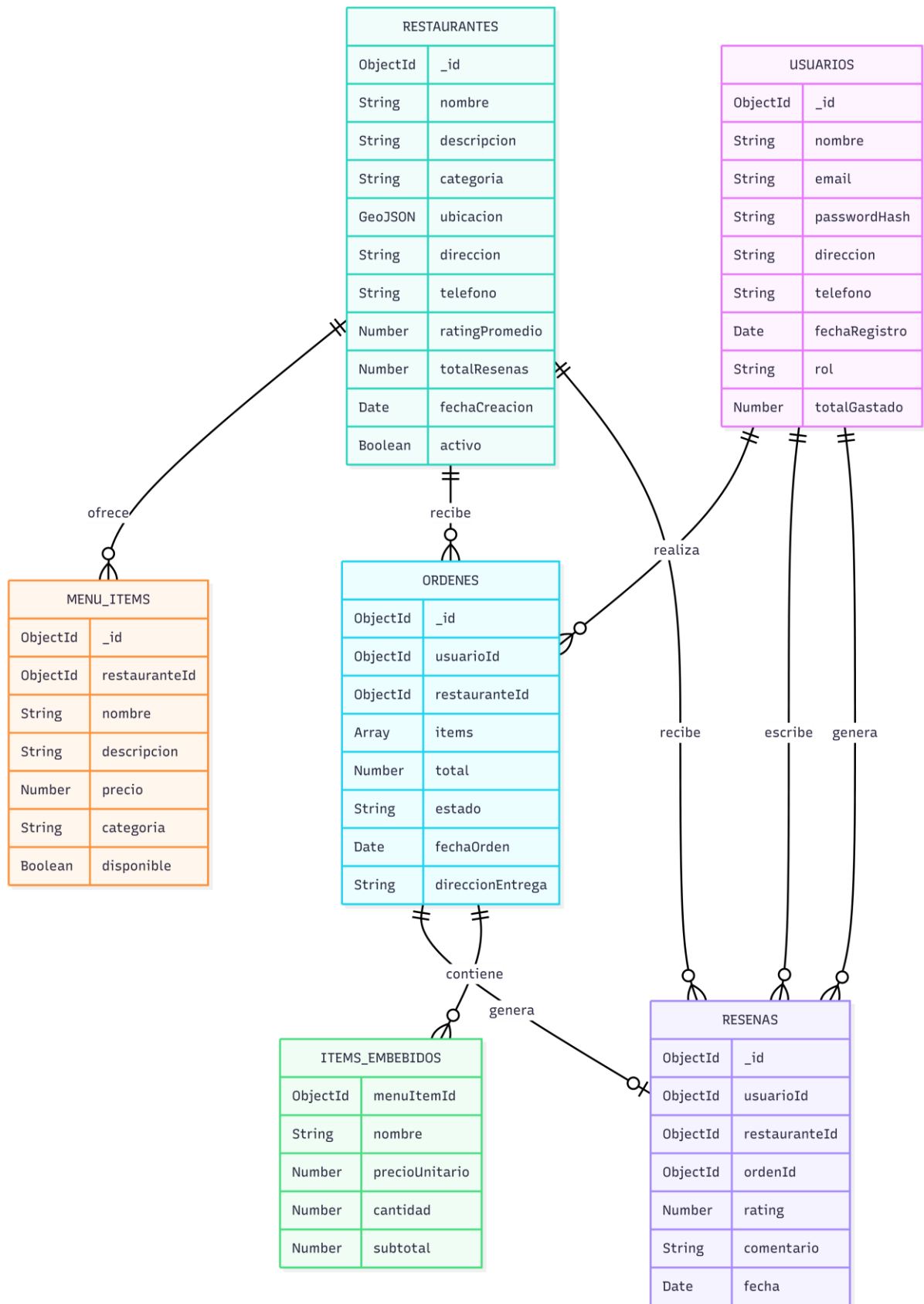
Grupo 12

```
    fechaOrden: Date,  
    direccionEntrega: String  
}
```

Colección: Reseñas

```
{  
  _id: ObjectId,  
  usuarioid: ObjectId,  
  restaurantId: ObjectId,  
  ordenId: ObjectId,  
  rating: Number,  
  comentario: String,  
  fecha: Date  
}
```

Grupo 12



Validación con JSON Schema

Objetivo de la validación

MongoDB permite definir reglas de validación estructural a nivel de colección mediante *\$jsonSchema*.

Esta funcionalidad se utilizará para:

- Garantizar integridad estructural de los documentos.
- Prevenir inserciones con tipos incorrectos.
- Forzar campos obligatorios.
- Implementar reglas básicas de negocio a nivel base de datos.
- Reducir errores provenientes del backend.

Dado que MongoDB es NoSQL y flexible por naturaleza, la validación permite mantener control estructural sin perder flexibilidad.

Estrategia de validación adoptada

Se decidió:

- Validar campos obligatorios críticos.
- Validar tipos BSON específicos.
- Aplicar restricciones numéricas (mínimos).
- Aplicar patrones regex (email).
- Validar estructura de subdocumentos embebidos.
- Aplicar enum en campos con estados controlados.

Validación - Colección Usuarios

Objetivo

- Garantizar unicidad estructural.
- Evitar registros sin email.
- Controlar tipo de datos.
- Evitar valores negativos en métricas acumuladas.

Ejemplo técnico

Se validará:

- nombre: obligatorio, string
- email: obligatorio, formato válido

Grupo 12

- fechaRegistro: obligatorio, date
- totalGastado: número ≥ 0
- rol: solo puede ser cliente o admin

Validación - Colección Órdenes

Esta es la colección más importante desde modelado.

Se validará:

- Debe tener al menos un item.
- Cada item debe tener cantidad ≥ 1 .
- Precio unitario ≥ 0 .
- Total ≥ 0 .
- Estado debe pertenecer a un conjunto controlado.

Reglas implementadas:

- Validación de array (bsonType: "array")
- Validación de subdocumentos internos
- Uso de minItems
- Uso de minimum
- Uso de enum para estado

Esto demuestra dominio de modelado documental avanzado.

Beneficios de la validación en el modelo

- Reduce corrupción de datos.
- Mantiene consistencia estructural.
- Protege la base de datos frente a errores de API.
- Permite que el modelo tenga reglas formales, no solo implícitas.
- Fortalece la arquitectura de datos desde la capa persistente.

Justificación de embedding vs referencing

El diseño del modelo documental se realizó bajo los principios de modelado orientado a consultas (query-driven design), considerando:

- Patrones de acceso a datos
- Cardinalidad de relaciones
- Ciclo de vida de los datos

Grupo 12

- Tamaño máximo de documento (16MB)
- Escalabilidad horizontal
- Frecuencia de actualización
- Aislamiento y atomicidad

Embedding: Items dentro de Órdenes

Decisión:

Los artículos comprados en una orden (items) se modelan como subdocumentos embebidos dentro de la colección ordenes.

Ejemplo conceptual:

```
{
  _id: ObjectId,
  usuarioid: ObjectId,
  restaurantId: ObjectId,
  items: [
    {
      menuItemId: ObjectId,
      nombre: "Pizza",
      precioUnitario: 80,
      cantidad: 2,
      subtotal: 160
    }
  ],
  total: 160
}
```

Justificación técnica

1. Ciclo de vida dependiente

Los items:

- No existen fuera de la orden.

Grupo 12

- No tienen sentido independiente.
- Se crean y eliminan junto con la orden.

En MongoDB, cuando el ciclo de vida es dependiente → Embedding es recomendado.

2. Atomicidad natural

MongoDB garantiza atomicidad a nivel documento.

Al embebidos:

- Crear orden + items ocurre en una sola operación.
- No se requieren transacciones multi-documento.
- Se reduce complejidad de consistencia.

Si fueran referenciados:

- Se necesitarían múltiples inserts.
- Aumenta riesgo de inconsistencias parciales.

3. Patrón de lectura dominante

Las consultas típicas serán:

- Ver detalle de una orden
- Mostrar historial completo de pedido
- Calcular total por orden

Siempre se necesita la orden junto con sus items.

Embedding evita:

- \$lookup
- joins
- múltiples consultas
- mayor latencia

4. Rendimiento de lectura

MongoDB está optimizado para:

- Leer documentos completos
- Minimizar joins

Embedding reduce:

Grupo 12

- Round trips
- I/O
- Overhead de red

5. Tamaño controlado del documento

Un orden promedio:

- 5–15 items
- Tamaño estimado muy inferior a 16MB

No existe riesgo real de superar límite de documento.

Cuando no usaríamos embedding aquí

- Si los items necesitaran actualizarse individualmente
- Si el número de items fuera extremadamente grande (ej. miles por orden)
- Si se requiriera acceso independiente frecuente

Ninguno de estos aplica en este sistema.

Referencing: Usuario ↔ Órdenes

Decisión:

Las órdenes contienen *usuarioid* como referencia.

Justificación Técnica

1. Alta cardinalidad

Un usuario puede tener:

- Decenas
- Cientos
- Miles de órdenes

Si embebiéramos órdenes dentro del usuario:

{

 _id: ObjectId,

 nombre: "Carlos",

 ordenes: [... miles ...]

Grupo 12

}

Problemas:

- Crecimiento ilimitado del documento
- Riesgo de alcanzar 16MB
- Documento caliente (hot document)
- Contención de escritura

2. Escalabilidad horizontal

Las órdenes crecen mucho más rápido que los usuarios.

Separarlas permite:

- Mejor distribución en sharding
- Escalar la colección ordenes independientemente
- Balanceo más eficiente

3. Frecuencia de actualización distinta

Usuarios:

- Se actualizan raramente

Órdenes:

- Se crean constantemente

Separarlas reduce:

- Contención
- Escrituras innecesarias

Referencing: Restaurante ↔ MenuItems

Decisión:

MenuItems es colección separada de *restaurantId*.

Justificación Técnica

1. Tamaño potencialmente grande

Grupo 12

Un restaurante puede tener:

- 50–500 platillos
- Cambios frecuentes de disponibilidad
- Actualizaciones individuales

Si embebiéramos:

- Cada cambio obligaría a reescribir el documento completo
- Documento crecería excesivamente

2. Consultas independientes

Se necesita:

- Buscar platillos por texto
- Filtrar por precio
- Buscar por categoría
- Ordenar por popularidad

Si estuvieran embebidos:

- No se podrían indexar eficientemente
- Las consultas serían más costosas

3. Índices específicos

Se requieren:

- Índice de texto
- Índice por precio
- Índice por restauranteld

Referencing permite indexación directa.

Referencing: Reseñas

Decisión:

reseñas es colección independiente.

Justificación Técnica

1. Alta cardinalidad

Grupo 12

Un restaurante puede tener:

- Miles de reseñas

Embedding implicaría:

- Documento gigante
- Problemas de crecimiento
- Reescritura constante

2. Actualización frecuente

Reseñas:

- Se agregan constantemente
- Se consultan por separado
- Se ordenan por fecha o rating

Colección separada permite:

- Paginación eficiente
- Índices dedicados
- Consultas analíticas optimizadas

3. Agregaciones eficientes

Para calcular:

- Promedio de rating
- Conteo total
- Tendencias temporales

Es más eficiente con colección independiente usando *\$group*.

Resumen:

Relación	Decisión	Razón principal
Orden → Items	Embedding	Ciclo de vida dependiente + atomicidad

Grupo 12

Usuario → Órdenes	Referencing	Alta cardinalidad + escalabilidad
Restaurante → MenuItems	Referencing	Consultas independientes + indexación
Restaurante → Reseñas	Referencing	Crecimiento alto + analítica

Método	Para qué sirve
insertOne()	Crear un documento individual
insertMany()	Crear muchos documentos
bulkWrite()	Operaciones masivas optimizadas
updateOne()	Actualizar un documento
updateMany()	Actualizar múltiples
deleteOne()	Eliminar uno
deleteMany()	Eliminar varios
aggregate()	Lookup y pipelines
faker	Generar datos realistas

- Faker
- insertMany en batches
- JSON Schema activo
- Datos referenciados reales
- Cálculo correcto de totales
- Distribución realista de estados

Reglas de integridad del modelo

Integridad de Entidades

- Cada usuario debe poseer un identificador único _id.
- Cada restaurante debe tener ubicación válida en formato GeoJSON.
- Cada orden debe pertenecer a un único usuario.
- Cada reseña debe estar asociada a un restaurante y opcionalmente a una orden.

Integridad Referencial (Lógica)

Grupo 12

MongoDB no impone integridad referencial automática, por lo tanto:

- El campo `usuarioid` en órdenes debe corresponder a un usuario existente.
- El campo `restauranteld` en reseñas debe existir en la colección `restaurantes`.
- El campo `ordenId` en reseñas debe existir en órdenes.

Estas validaciones serán manejadas en la capa de aplicación.

Restricciones de Dominio

- $\text{Rating} \in [1, 5]$
- Cantidad de item ≥ 1
- $\text{Precio} \geq 0$
- $\text{Total} \geq 0$
- $\text{Estado} \in \{\text{pendiente}, \text{en_proceso}, \text{entregado}, \text{cancelado}\}$
- Restaurante activo debe ser `true` para permitir nuevas órdenes.

Integridad Estructural

- Una orden debe contener al menos un item.
- Los items no pueden existir fuera de una orden.
- No se permitirá `embedding` ilimitado.
- Reseñas no serán embebidas para evitar crecimiento no controlado.

Consistencia Derivada

Algunos campos son derivados:

- `ratingPromedio` en restaurante.
- `totalResenas` en restaurante.
- `totalGastado` en usuario.

Estos deben recalcularse mediante operaciones controladas para evitar inconsistencias.

Descripción de transacciones implementadas

En este sistema se definieron dos escenarios críticos donde la atomicidad es indispensable.

Transacción 1: Creación de una Orden

Cuando un usuario realiza un pedido, se ejecutan múltiples operaciones que deben completarse simultáneamente:

Grupo 12

1. Insertar el documento en la colección ordenes
2. Verificar disponibilidad de los artículos seleccionados
3. Actualizar el inventario o disponibilidad de dichos artículos
4. Registrar la acción

Estas operaciones se ejecutarán dentro de una sesión utilizando transacciones de MongoDB. Si cualquiera de las operaciones falla, se realizará rollback completo para garantizar que el sistema no quede en un estado inconsistente.

Transacción 2: Eliminación Controlada de Restaurante

En caso de eliminar un restaurante, se deben considerar entidades relacionadas:

- Artículos del menú asociados
- Reseñas asociadas
- Órdenes pendientes

Se propone realizar una transacción que marque el restaurante como inactivo, desactive los artículos asociados y actualice el estado de órdenes pendientes a cancelado. Esto evita referencias huérfanas y mantiene coherencia lógica.

Explicación detallada de aggregation pipelines

El sistema incorpora consultas analíticas complejas utilizando el Aggregation Framework de MongoDB, permitiendo generar reportes empresariales y métricas operativas sin necesidad de procesos externos. Optimizando primero mediante \$match y aprovechando los índices definidos previamente.

A continuación se describen los principales pipelines analíticos del sistema

Pipeline 1: Top 10 platillos más vendidos

1. \$match: Filtra por restaurante y estado entregado.
2. \$unwind: Descompone el arreglo items.
3. \$group: Suma cantidad y subtotal por producto.
4. \$lookup: Obtiene datos del menú.
5. \$sort y \$limit: Ordena y limita resultados.

Pipeline 2: Ventas mensuales

Grupo 12

1. \$match: Filtra órdenes entregadas.
2. \$group: Agrupa por mes usando \$dateTrunc.
3. \$project: Calcula ticket promedio.
4. \$sort: Orden cronológico.

Pipeline 3: Restaurantes mejor calificados

1. \$group: Calcula promedio y conteo de rating.
2. \$lookup: Une información del restaurante.
3. \$sort: Ordena por promedio descendente.

Índices definidos y análisis de rendimiento con explain()

El diseño del sistema se realizó considerando que todas las consultas del backend deberán utilizar índices, evitando escaneos completos de colección (COLLSCAN), esto con el fin de reducir latencia, disminuir carga en memoria y permitir el crecimiento de la colección de órdenes. Se validará el uso de índices mediante el método .explain("executionStats") en donde se verificará que el plan de ejecución utilice IXSCAN en lugar de COLLSCAN.

Índice simple y único en usuarios:

{ email: 1 }

Optimiza autenticación y evita registros duplicados.

Índice compuesto en ordenes:

{ restaurantId: 1, estado: 1, fechaOrden: -1 }

Optimiza consultas del panel de pedidos y permite resolver el ordenamiento por fecha directamente desde el índice.

Índice multikey en ordenes:

{ "items.menuItemId": 1 }

Permite consultas eficientes sobre arreglos embebidos, especialmente en análisis

de productos más vendidos.

Índice de texto en restaurantes:

```
{ nombre: "text", descripcion: "text", categoria: "text" }
```

Permite búsquedas por palabras clave sin escaneo completo.

Validación mediante explain("executionStats"):

Se analizarán los siguientes indicadores:

- winningPlan debe mostrar uso de IXSCAN
- No debe aparecer COLLSCAN
- totalDocsExamined debe ser cercano a nReturned
- totalKeysExamined debe reflejar uso efectivo del índice
- executionTimeMillis debe mantenerse bajo

Ejemplo:

```
db.ordenes
```

```
.find({ restaurantId: ObjectId("ID"), estado: "entregado" })
```

```
.sort({ fechaOrden: -1 })
```

```
.explain("executionStats")
```

Creación de documentos

Documento embebido: Surge al momento de crear una orden, cuando el usuario selecciona varios artículos:

```
items: [
```

```
{ articuloId, cantidad, precioUnitario }
```

```
]
```

Los items se guardan embebidos dentro del documento orden.

Documento referenciado: Surge cuando se crea una reseña vinculada a una orden o cuando se crea una orden vinculada a un usuario. Para crear uno o varios documentos: insertOne() e insertMany()

Lectura y consulta

- Filtros: Mostrar órdenes con estado determinado (ENTREGADO, CANCELADO)
- Proyecciones: Para que se muestren total y fecha de algún pedido o para mostrar del listado de artículos solo precio y nombre
- Ordenamiento: Cuando el historial se ordena por fecha descendente
- Skip: Implementación de paginación (página 2, página 3)
- Limit: Para mostrar solo 10 restaurantes u órdenes por página

Lookup: Para consultar una orden con datos del usuario relacionado o para mostrar restaurantes con sus reseñas.

Actualización

Actualizar 1 documento: Para actualizar el estado de una orden, por ejemplo, en proceso a entregado

Actualizar varios documentos: Actualizar estado de restaurantes, para indicar si está abierto o cerrado.

Eliminación

Eliminar 1 documento: Cuando un usuario elimina una reseña

Eliminar varios documentos: Eliminar reseñas, pedidos o información relacionada a determinado usuario.

GRIDFS y archivos

Se utilizará para almacenar los items del menú en formato JSON de cada restaurante. La imagen se guarda en GridFS y el artículo solo guarda el ID del archivo

Colección artículos

{

Grupo 12

```
"_id": ObjectId(),  
"nombre": "Pizza Pepperoni",  
"precio": 85,  
"imagenId": ObjectId("gridfsFileId")  
}
```

Se propone utilizar `bulkwrite()` para inserción masiva inicial de 50,000 órdenes, actualización masiva de estados de pedidos antiguos e inserción masiva de artículos.

El uso de operaciones bulk reduce significativamente la cantidad de viajes al servidor y mejora el rendimiento general del sistema, especialmente en operaciones de gran volumen.

Agregaciones y otros

Simple: `countDocuments()` para contar cuántas ordenes hay por estado, `distinct` para categorías de la comida (Bebida, postre, pizza).

Complejas: pipelines para restaurante con mayor cantidad de ventas, artículo más vendido, ventas totales por mes, promedio de gasto por usuario.

Se utilizarán:

- `$unwind`
- `$group`
- `$sum`
- `$avg`
- `$sort`
- `$lookup`

Manejo de arrays: Para agregar artículos a una orden (`$push`), Eliminar artículo de una orden (`$pull`), agregar restaurante a favoritos sin duplicado (`$addToSet`)

Manejo de documentos embebidos: Se actualiza la cantidad de un artículo específico dentro del array `items` usando operador posicional.

Propuesta de shard key y análisis de escalabilidad

Grupo 12

Dado que la colección ordenes será la de mayor crecimiento, se propone aplicar sharding sobre dicha colección.

Colección: Órdenes

Shard Key: usuariold

Justificación:

- Alta cardinalidad
- Distribución uniforme esperada
- Las consultas más frecuentes se realizan por usuario (historial de pedidos)
- Reduce riesgo de hotspot en comparación con una clave basada únicamente en fecha

Este diseño permite escalar horizontalmente agregando nuevos shards sin modificar el modelo de datos.

BI Connector con Tableau

Se conectará MongoDB Atlas con Tableau en donde podríamos tener algunos dashboards propuestos como ventas mensuales, restaurantes mejor calificados, Platos más vendidos, usuarios top por gasto u órdenes por estado. Todo lo anterior nos permitirá realizar un análisis empresarial sobre datos NoSQL y asimismo, comprender el comportamiento de los datos y poder visualizar de mejor manera información sobre los restaurantes.